Úvod do grafových algoritmů.

Orientovaný, neorientovaný graf. BFS. DFS.

Tomáš Bayer | bayertom@fsv.cvut.cz

Katedra geomatiky, fakulta stavební ČVUT.

Obsah přednášky

1. Graf
   - Neorientovaný graf
   - Orientovaný graf
2. Spojová reprezentace grafů
3. Prohledávání grafu
   - Prohledávání grafu do šířky
   - Prohledávání grafu do hloubky

1. Graf (1/2)

Graf: datová struktura popisující vztahy mezi objekty.

Datová struktura tvořená uzly U a hranami H.

Zpravidla konečný počet $\Rightarrow$ konečný graf.

Topologicko/geometrický model skutečnosti.

Informace o poloze méně důležitá než vzájemný vztah.

Lze znázornit nekonečně mnoha způsoby.

Široké využití v mnoha oblastech:

Úlohy o dopravním spojení, logistické problémy, optimální trasa, plánování, navigace, propustnost sítě, přenos energie, komprese dat.

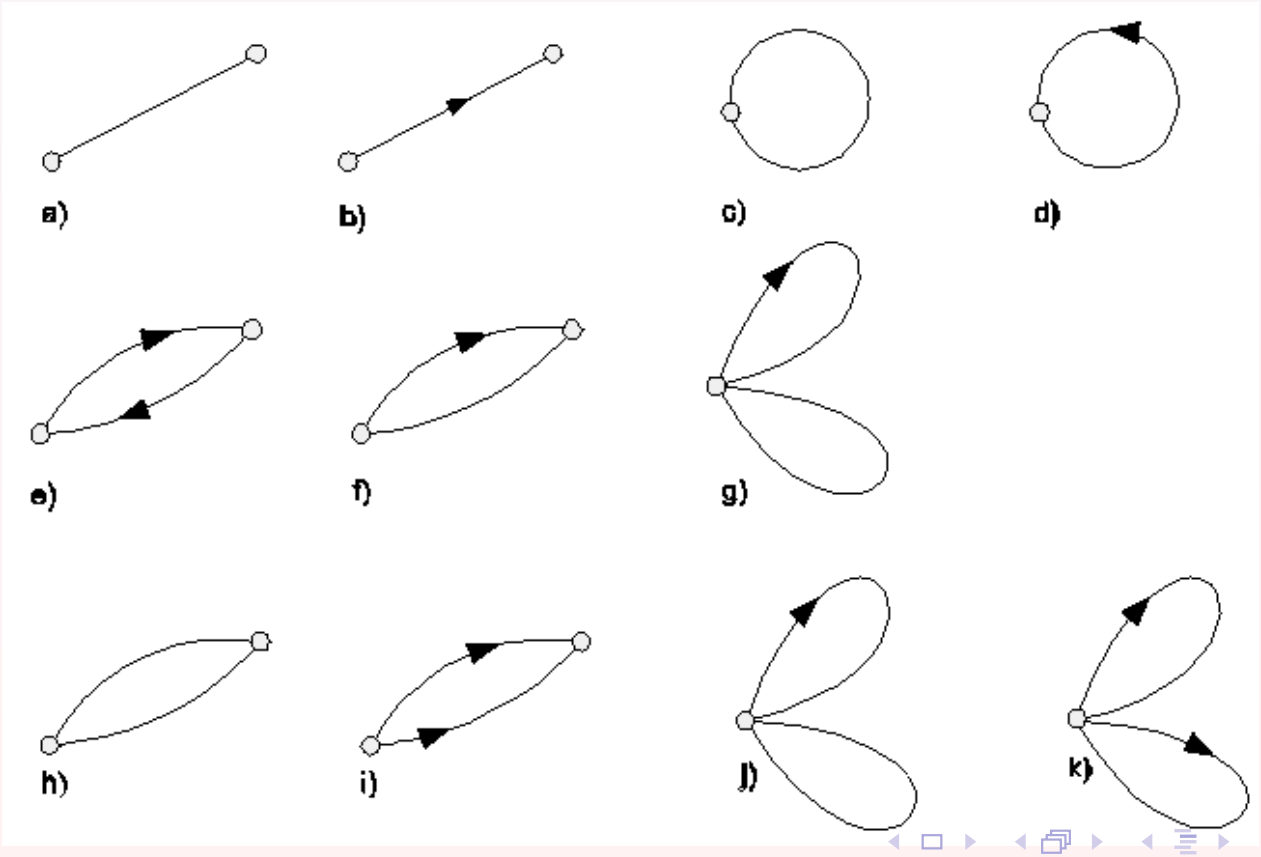
Dělení grafů:

- neorientované,
- orientované,
- částečně orientované.

2. Graf (2/2)

Datová struktura tvořená uzly U a hranami H.
Zpravidla konečný počet $\Rightarrow$ konečný graf.
Lze znázornit nekonečně mnoha způsoby.

Přímá neorientovaná hrana (a), orientovaná hrana (b), neorientovaná smyčka (c), orientovaná smyčka (d), rovnoběžné hrany (e, f), rovnoběžná hrana se smyčkou (g), násobné hrany (h, i), násobné hrany se smyčkou (j, k).



3. Neorientovaný graf

Uspořádaná trojice disjunktních množin

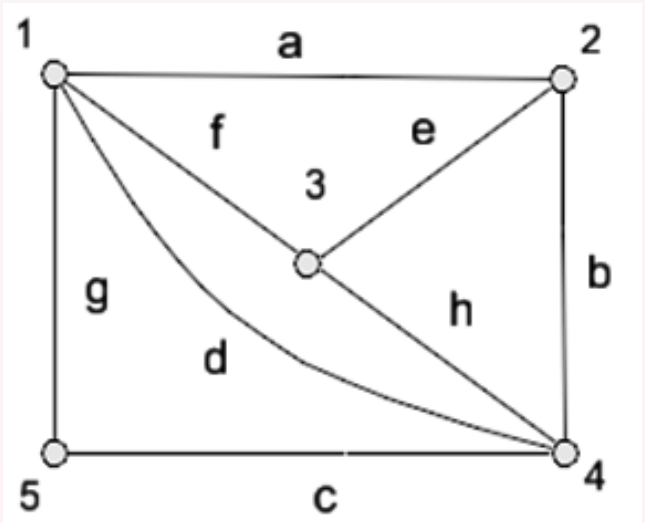
$$G = \langle H, U, \rho \rangle ,$$

kde H představují hrany, U uzly, ρ incidenci grafu G.

Incidence

$$\rho : H \to U \otimes U,$$

přiřazuje každé hraně z množiny H neprázdnou množinu dvojic uzlů z množiny U.



Množina uzlů: $U = \{1, 2, 3, 4, 5\}$,

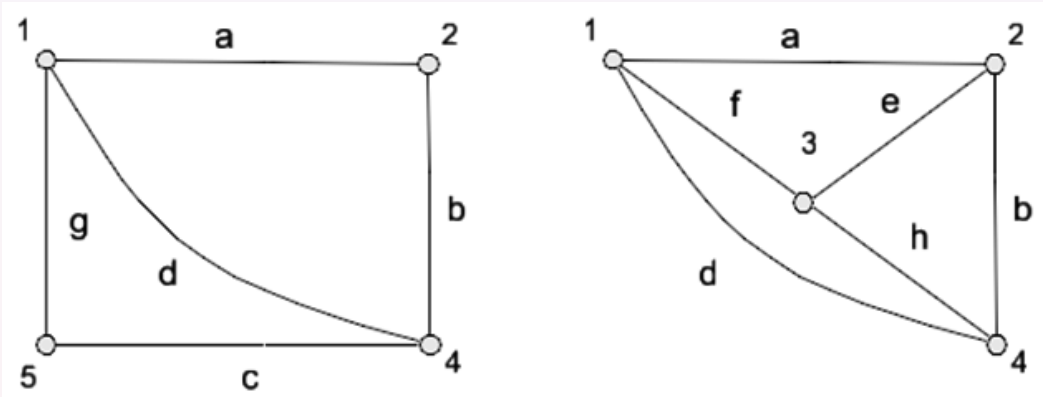
Množina hran: $H = \{a, b, c, d, e, f, g, h\}$,

Incidence: $\rho(a) = \{1, 2\}$, $\rho(b) = \{2, 4\}$, $\rho(c) = \{4, 5\}$, $\rho(d) = \{1, 4\}$, $\rho(e) = \{2, 3\}$, $\rho(f) = \{1, 3\}$, $\rho(g) = \{1, 5\}$, $\rho(h) = \{3, 4\}$.

4. Podgraf a faktor

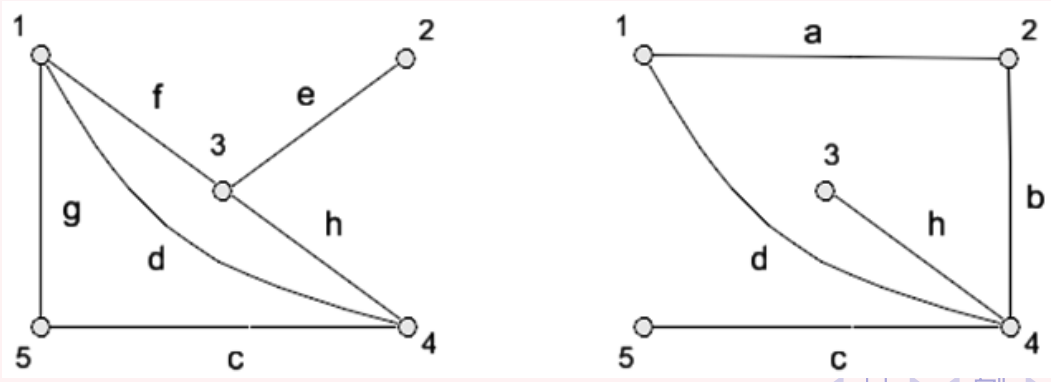
Podgraf $G' \subset G$:

$G' = (H', U', \rho')$ je podgraf $G = (H, U, \rho)$, platí–li $H' \subset H$ a $U' \subset U$ a pro každé $h \in H'$ je $\rho'(h) \subset \rho(h)$.



Faktor grafu:

Podgraf G', jehož množina uzlů U' je totožná s množinou uzlů U grafu G.



5. Sled, tah, cesta

Sled: uspořádaná posloupnost uzlů a hran,

$$\langle 1, a, 2, e, 3, f, 1, d, 4, b, 2, e, 3, h, 4, c, 5 \rangle .$$

Tah: sled, ve kterém se vyskytuje každá hrana nejvýše jednou,

$$\langle 1, a, 2, e, 3, f, 1, g, 5, c, 4, b, 2 \rangle .$$

Uzavřený tah: začíná a končí ve stejném uzlu.

Cesta: tah, ve kterém se každý uzel vyskytuje nejvýše jednou,

$$\langle 3, f, 1, a, 2, b, 4, c, 5 \rangle .$$

Kružnice: uzavřená cesta, začíná a končí ve stejném uzlu,

$$\langle 3, f, 1, g, 5, c, 4, b, 2, e, 3 \rangle .$$

Stupeň uzlu $d(u)$:

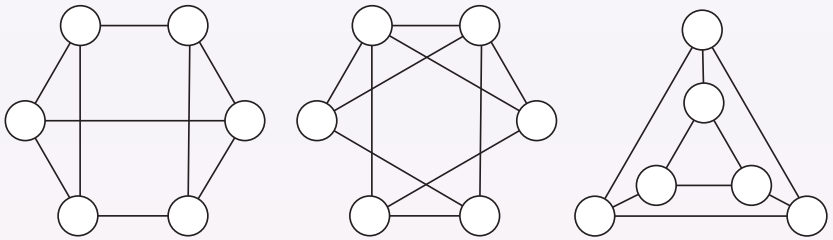
Počet hran, které incidují s uzlem u, důležitá sudost, lichost.

$$\sum_{u \in G} d(u) = 2 \, \|H\|$$

Příklady: $d(1) = 2$, $d(5) = 1$.

6. *Grafická reprezentace grafu

Graf lze reprezentovat mnoha způsoby: nakreslení.



Shoda grafů:

Grafy $G_1 = \langle H_1, U_1, \rho_1 \rangle$, $G_2 = \langle H_2, U_2, \rho_2 \rangle$.

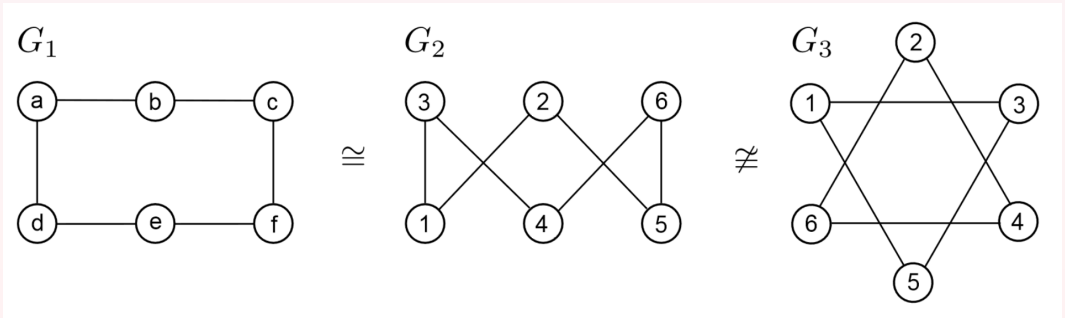
Platí, že $G_1 = G_1 \Leftrightarrow H_1 = H_2$, $U_1 = U_2$, $\rho_1 = \rho_2$.

Izomorfismus G_1, G_2:

Důležité při posuzování, zda G_1, G_2 "identické" (byť různě nakreslené).

$G_1 \cong G_2 \Leftrightarrow \exists$ jednoznačné zobrazení $f : G_1 \to G_2$ takové, že pro libovolné $h \in H$ platí

$$\rho_1(h) = [u, v] \in G_1 \exists \rho_2(f(h)) = [f(u), f(v)] \in G_2.$$



$f : a \to 3, b \to 4, c \to 6, f \to 5, e \to 2, d \to 1.$

7. Orientovaný graf

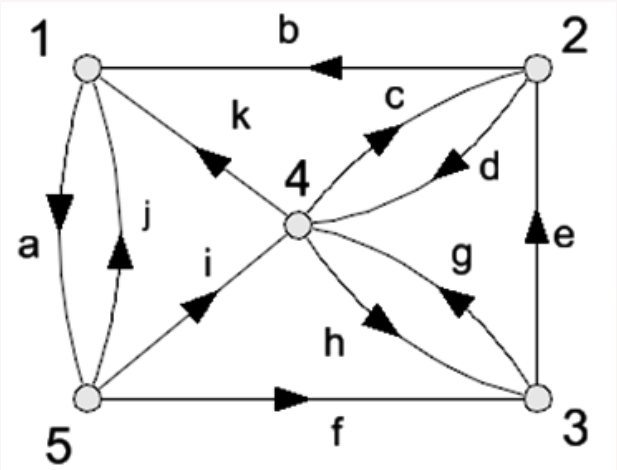
Uspořádaná trojice disjunktních množin $G = \langle H, U, \sigma \rangle$, kde

$$\sigma : H \to U \times U.$$

Pro $h \in H$

$$\sigma(h) = \{u, v\},$$

kde u je počáteční a v koncový uzel.



Stupeň uzlu: suma vstupujících $d^+(u)$ a vystupujících hran $d^-(u)$

$$d(u) = d^+(u) + d^-(u).$$

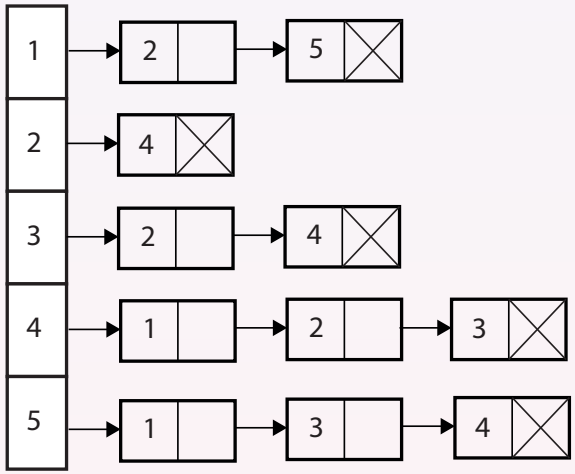
8. Reprezentace grafů

Maticová reprezentace:

Výhodná pro výpočty, u řídkých grafů mnoho nulových prvků.

Neefektivní, velké paměťové nároky.

Lze vylepšit s použitím řídkých matic.



Reprezentace spojovým seznamem :

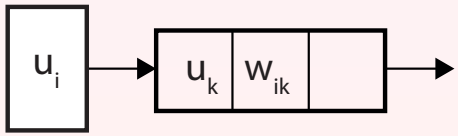
Ukládáme jen uzly, mezi kterými jsou vztahy, menší paměťové nároky.

Méně přehledné pro rozsáhlé grafy.

V praxi použit 2D seznam: 1D seznam, každá položka opět 1D seznam.

Ohodnocené grafy:

Přidáno ohodnocení w_{ik} hrany u_{ik}.



9. Spojová reprezentace v Pythonu

Spojová reprezentace grafu prostřednictvím Dictionary

```
key: value
```

Klíčem uzel, hodnotou vnořený seznam incidujících uzlů.

```
V1 : [V1, V2, ..., Vk]
```

Obecný tvar reprezentace:

```
G={
   V1 : [V1, V2, ..., Vk],
   V2 : [V1, V2, ..., Vk],
   ...
   Vk : [V1, V2, ..., VK]
  }
```

Grafy s ohodnocením:

```
G={
   V1 : {V1:W1, V2:W2, ..., Vk:Wk},
   V2 : {V1:W1, V2:W2, ..., Vk:Wk},
   ...
   Vk : {V1:W1, V2:W2, ..., Vk:Wk}
  }
```

10. Prohledávání grafu

Systematické procházení hran a uzlů grafu dle zadané strategie.

Nejčastěji řešené úlohy:

1. Dostupnost uzlu v z u.
2. Množina všech uzlů dostupných z u.
3. Nalezení cesty z u do v (libovolná, optimální).
4. Nalezení cesty z u do všech dosažitelných vrcholů.

Aplikace v oblasti geoinformatiky, logistiky, dopravy, navigace.

Techniky prohledávání grafu:

- Prohledávání do šířky.
- Prohledávání do hloubky.
- Kombinace + heuristiky.

11. Prohledávání grafu do šířky

BFS (Breadth First Search).

Častá strategie pří procházení grafu.

Vychází z něj: hledání minimální kostry, nejkratší cesty.

Lze použít pro neorientované/orientované grafy.

Idea:

- Postupné rozhlížení z uzlu u do všech jeho doposud neprohledaných sousedů v.
- Získáme uzly v jsou z tohoto uzlu dostupné.
- Následně skok na prvního souseda u.
- Opakování postupu pro dosud neprohledané uzly u.

Výsledek:

Strom hledání do šířky: BF strom.

- Seznam všech uzlů dosažitelných z výchozího uzlu.
- Cesta do těchto uzlů tvořená minimem hran.

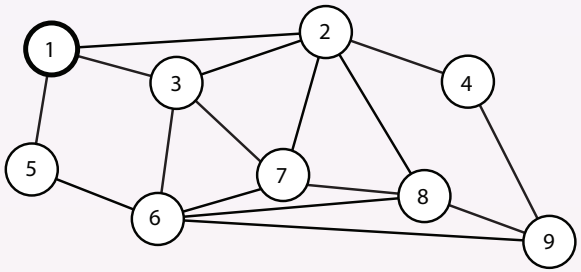
BF strom uložen jako P-strom, strom předchůdců.

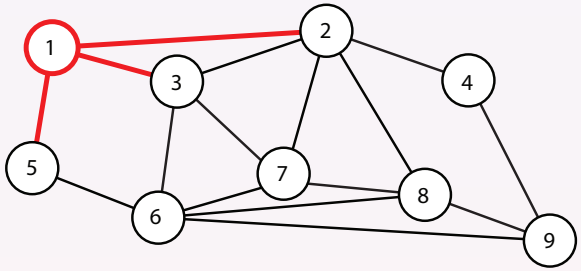
Velmi efektivní reprezentace.

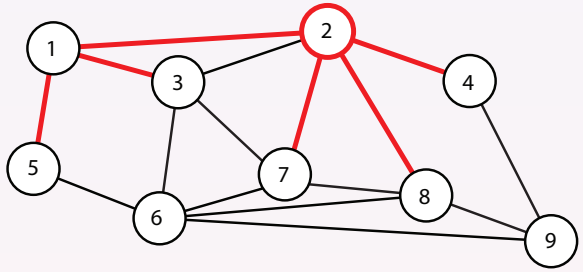
Složitost $O(\|U\| + \|H\|)$.

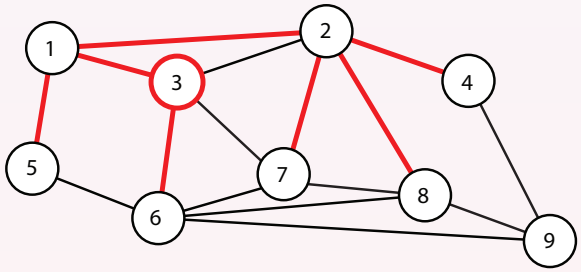
12. Ukázka BFS

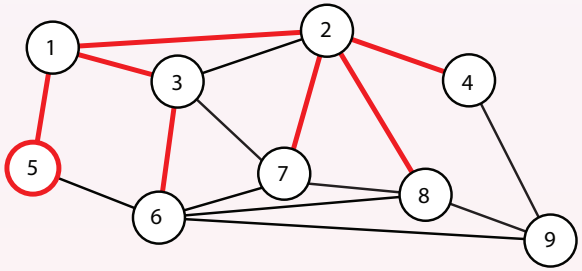
Startovní uzel: $u = 1$.

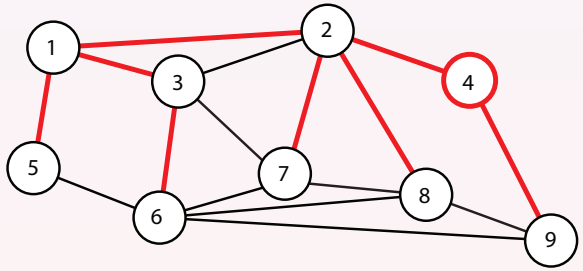


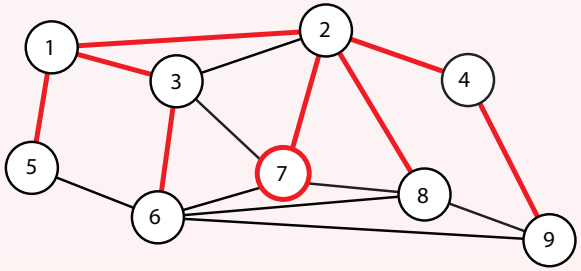


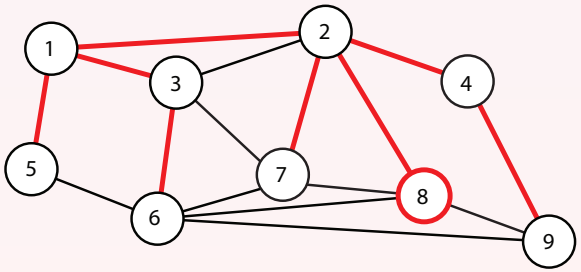


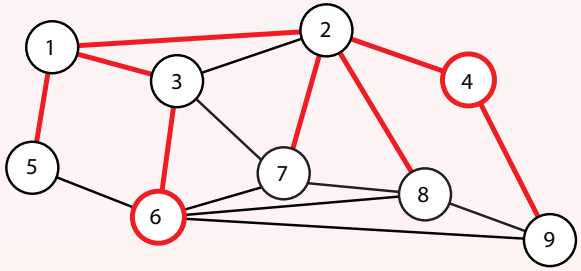












13. Rozdělení uzlů

Provádíme značkování uzlů do kategorií.

Aby bylo jasné, které lze ještě použít, a které již nikoliv.

3 kategorie uzlů:

- *Nové uzly (New, White)*
  Dosud neobjevené uzly.
  Uzel, na který narazíme poprvé.
- *Otevřené uzly (Open, Gray)*
  Již objevený uzel.
  Prozkoumáni někteří sousedé.
- *Uzavřené uzly (Closed, Black)*
  Již objevený uzel.
  Prozkoumáni všichni sousedé.

Grafy neobsahující cykly: není třeba značkovat uzly.

14. Princip BFS

Nezohledňuje ohodnocení hran grafu: všechny jednotkové.

Na začátku všechny uzly “N”, následně “O”, potom “C”.

Uzly a atributem “O” uloženy v Q.

Princip BFS:

- Z fronty Q vezmi aktuální uzel u .
- Pro každé v sousedící s u opakuj následující kroky:
  Pokud má v atribut “N”, změň ho na “O”.
  Vytvoř novou hranu $\{u, v\}$ a přidej ji do BF stromu.
- Poté změň stav u na “C”.

Místo přidávání $\{u, v\}$ do BF stromu lze využít *předchůdce uzlu v.*

Předchůdce uzlu.

Uzel u je předchůdcem p uzlu v

$$p(v) = u.$$

Na základě znalosti $p(v)$ lze cestu zpětně zrekonstruovat (netřeba přidávat $\{u, v\}$ do BF).

Rekonstrukce cesty od uzlu v ke kořenu u

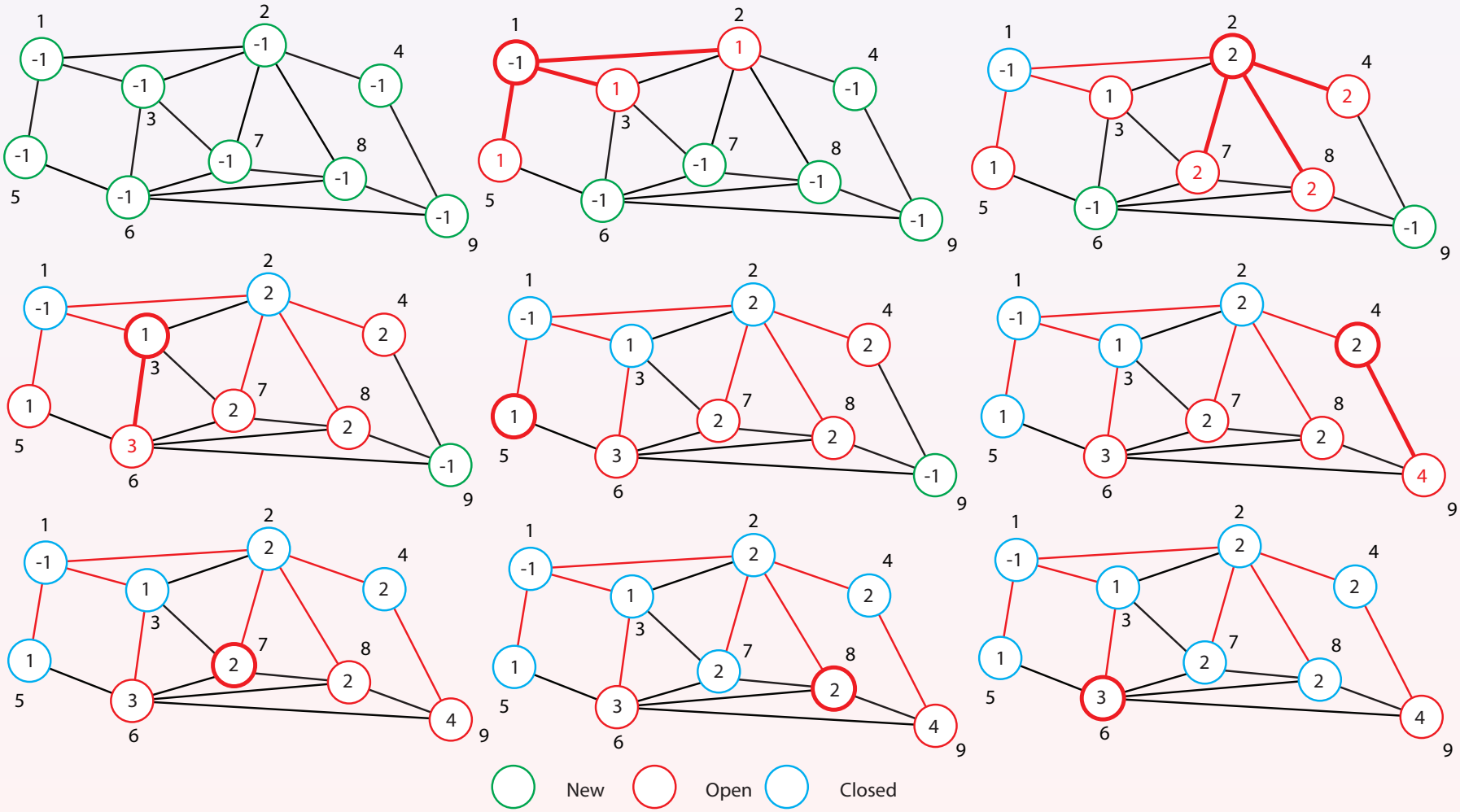
$$\langle v, p(v), p(p(v)), \ldots, u \rangle .$$

Pomocné proměnné.

BFS využívá 3 pomocné proměnné:

- stav uzlu $s[u]$,
- předchůdce $p[u]$ uzlu u,
- značku $d[u]$ při objevení uzlu (“vzdálenost” od u).

15. Princip BFS, ukázka



16. Implementace BFS

```
def BFS(G, start):
    s = ['N'] * (len(G)+1)      #All vertices are new
    p = [None] * (len(G)+1)     #Without predecessors
    Q = []                      #Empty queue
    Q.append(start)             #Add start vertex to Q
    s[u] = 'O'                  #Set as open
    while Q:
        u = Q.pop(0)            #Pop first node
        for v in G[u]:          #Browse incident nodes
            if s[v] == 'N':     #We found a new node
                s[v] = 'O'      #Change status to Open
                p[v] = u        #Set predecessors
                Q.append(v)     #Add v to Q
        s[u] = 'C'              #Change status
```

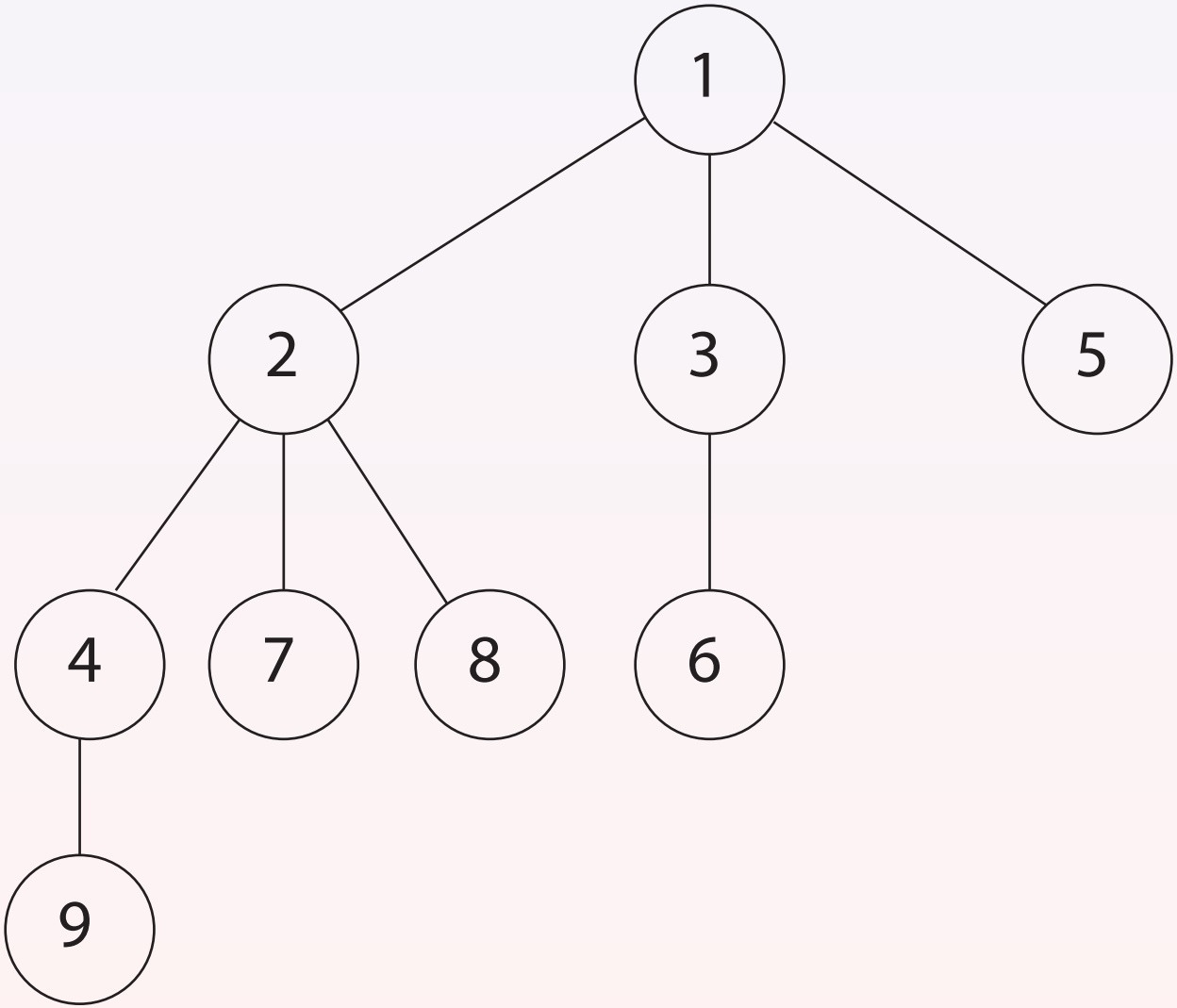
17. Ukázka činnosti algoritmu, $u = 1$

Aktualizace předchůdců a stav Q.

#	u_1	u_2	u_3	u_4	u_5	u_6	u_7	u_8	u_9	Q	$Q \leftarrow$
1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1,	2,3,5
2	-1	1	-1	-1	-1	-1	-1	-1	-1	2, 3, 5	4,7,8
3	-1	1	1	-1	-1	-1	-1	-1	-1	3, 5, 4, 7, 8	6
4	-1	1	1	-1	1	-1	-1	-1	-1	5, 4, 7, 8, 6	
5	-1	1	1	2	1	-1	-1	-1	-1	4, 7, 8, 6	9
6	-1	1	1	2	1	-1	2	-1	-1	7, 8, 6, 9	
7	-1	1	1	2	1	-1	2	2	-1	8, 6, 9	
8	-1	1	1	2	1	3	2	2	-1	6, 9	
9	-1	1	1	2	1	3	2	2	4	9	

Obsah Q: procházení BF stromu po úrovních.

18. Výsledný BF strom



19. Graf a jeho popis

Popis grafu:

```
G = {
    1 : [2, 3, 5],
    2 : [1, 3, 4, 7, 8],
    3 : [1, 2, 6, 7],
    4 : [2, 9],
    5 : [1, 6],
    6 : [3, 5, 7, 8, 9],
    7 : [2, 3, 6, 8],
    8 : [2, 6, 7, 9],
    9 : [4, 6, 8]
}
```

Výsledky:

```
print(p)
print(d)
print(s)
>>[None, 1, 1, 2, 1, 3, 2, 2, 4]
>>[0, 1, 1, 2, 1, 2, 2, 2, 3]
>>['C', 'C', 'C', 'C', 'C', 'C', 'C', 'C', 'C']
```

20. Zpětná rekonstrukce cesty

Cesta mezi uzly $\langle u, v \rangle$.

Postup od koncového uzlu v blížíme k počátečnímu uzlu u.

Využití předchůdce $p[v]$.

Opakujeme, dokud $v \neq u$.

```
def createPath(u, v, p):
    path = []                       //Create empty path
    while v != u and v != -1:       //Until we reach a start node
        path.append(v)              //Add current node
        v = p[v]                    //Go to the predeccessor
    path.append(v)                  //Add last vertex
    return path
```

Volání:

```
createPath(1, 9, p)
>> 1 2 4 9
```

Existuje i rekurzivní implementace.

Založena na opakovaném zkracování cesty.

21. Prohledávání grafu do hloubky

DFS (Depth First Search)

Prohledávání grafu s cílem dostat se do grafu co nejhlouběji.

Postupné procházení všech možných cest grafem.

Analogie bludiště: vracíme se zpět, když cesta nepokračuje.

Strategie známa jako *Backtracking*.

Jedna ze základních metod procházení stavového prostoru.

Idea:

- Z uzlu u jdeme do prvního dosud neprohledaného souseda v.
- Pokud takový uzel v neexistuje, návrat do uzlu, ze kterého jsme vstoupili do u, tj. do $p[u]$.
- Opakováno, dokud neobjeveny všechny dosažitelné uzly z výchozího uzlu.

Výsledek:

Strom hledání do šířky: DF strom.

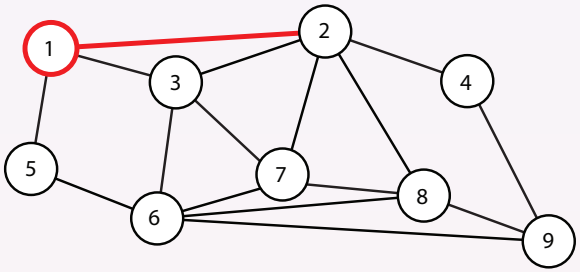
- Seznam všech uzlů dosažitelných z výchozího.
- Cesta do těchto uzlů není nejkratší.

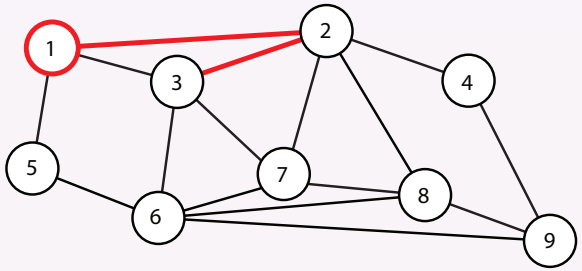
DF strom uložen jako P-strom, strom předchůdců.

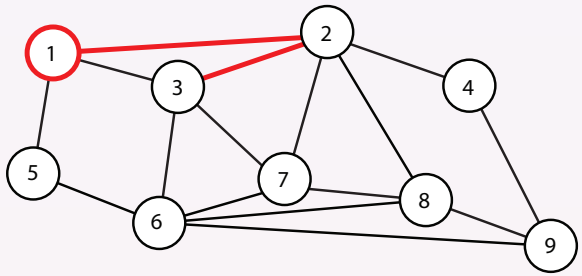
Složitost $O(\|U\| + \|H\|)$.

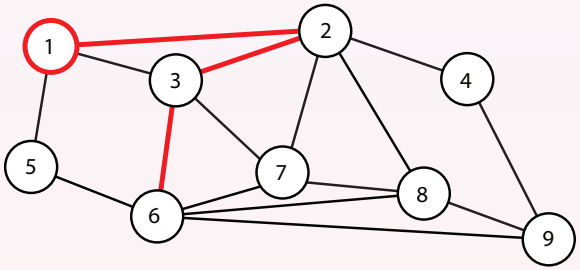
22. Ukázka DFS

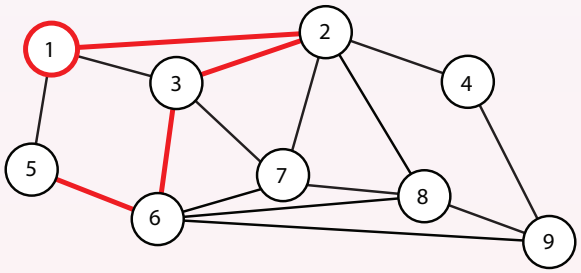
Startovní uzel $u = 1$.

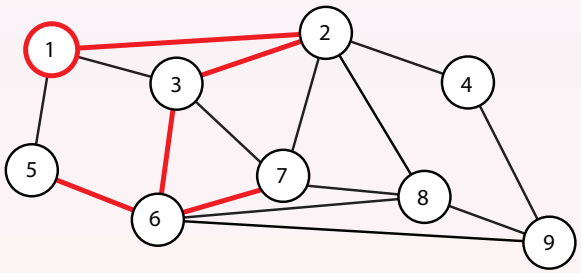


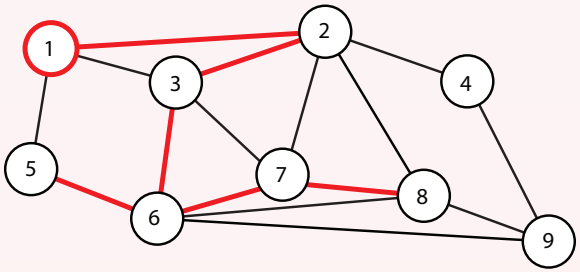


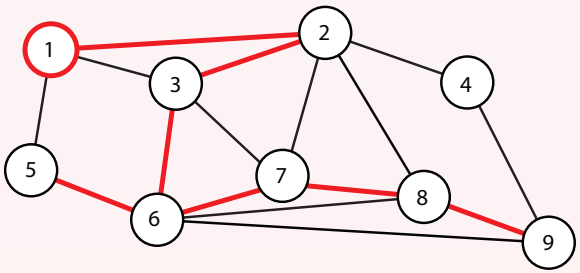


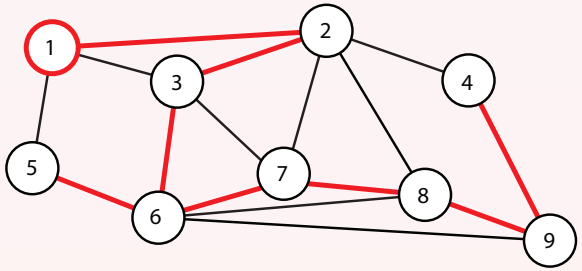












23. Princip DFS

Uchovávané informace o uzlu: stav uzlu $s[u]$, předchůdce $p[u]$ uzlu u.

Pro nový uzel u projdi všechny jeho sousedy v.

- nastavení předchůdce $p[v] = u$,
- rekurzivně projdi všechny sousedy uzlu v.
- na rozdíl od BFS uzel neoznačujeme jako Open.

Implementace: Iterativní, rekurzivní.

Iterativní implementace:

Fronta nahrazena zásobníkem.

Uzly procházeny v *opačném* směru (avšak není nutné).

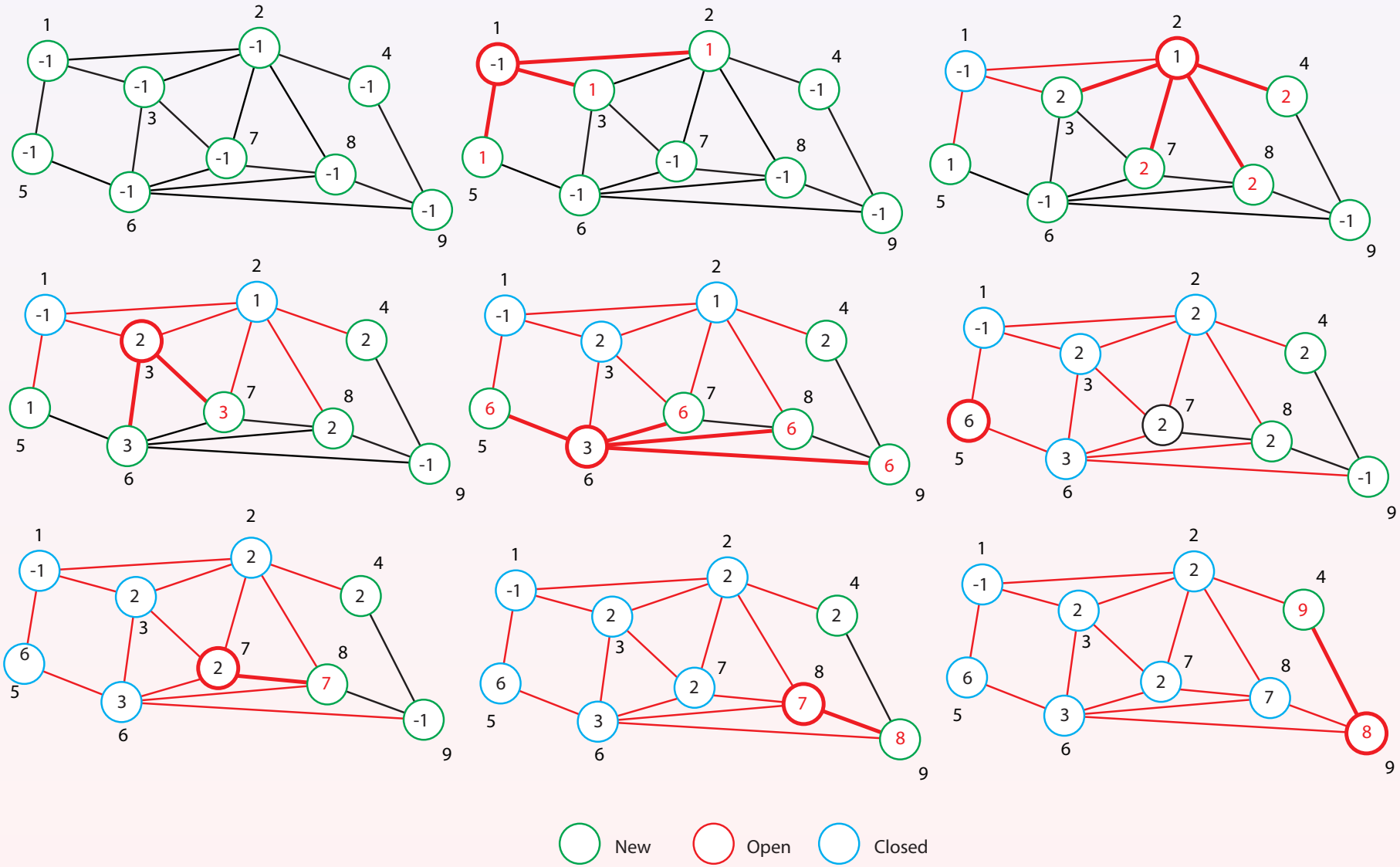
Rekurzivní implementace:

Procedura DFSR spuštěna 1x nad každým novým uzlem.

Pokud G souvislý, stačí 1x nad libovolným uzlem.

Výsledkem DF les: tvořen DF stromy (>, pokud G nesouvislé).

24. Princip DFS, ukázka



25. Iterativní implementace DFS

```
def DFS(G, start):
    s = ['N'] * (len(G)+1)          #All vertices are new
    p = [None] * (len(G)+1)         #Without predecessors
    S = []                          #Empty stack
    S.append(start)                 #Add start vertex to S
    while S:
        u = S.pop()                 #Pop node
        if s[u] == 'N':             #Is node new
            s[u] = 'O'              #Set as open
            S.append(u)
            for v in reversed(G[u]): #Browse incident nodes
                if s[v] == 'N':     #We found a new node
                    p[v] = u        #Set predecessor
                    S.append(v)     #Add v to Q
        s[u] = 'C'                  #Change status
```

Využívá zásobník.

26. Rekurzivní implementace DFS

Souvislý graf, 1x z libovolného uzlu:

```
def DFS(G, u):
    s = ['N'] * (len(G)+1)          #All vertices are new
    p = [None] * (len(G)+1)         #Without predecessors
    DFSR(G, s, p, u)                #Run DFS for connected graph
```

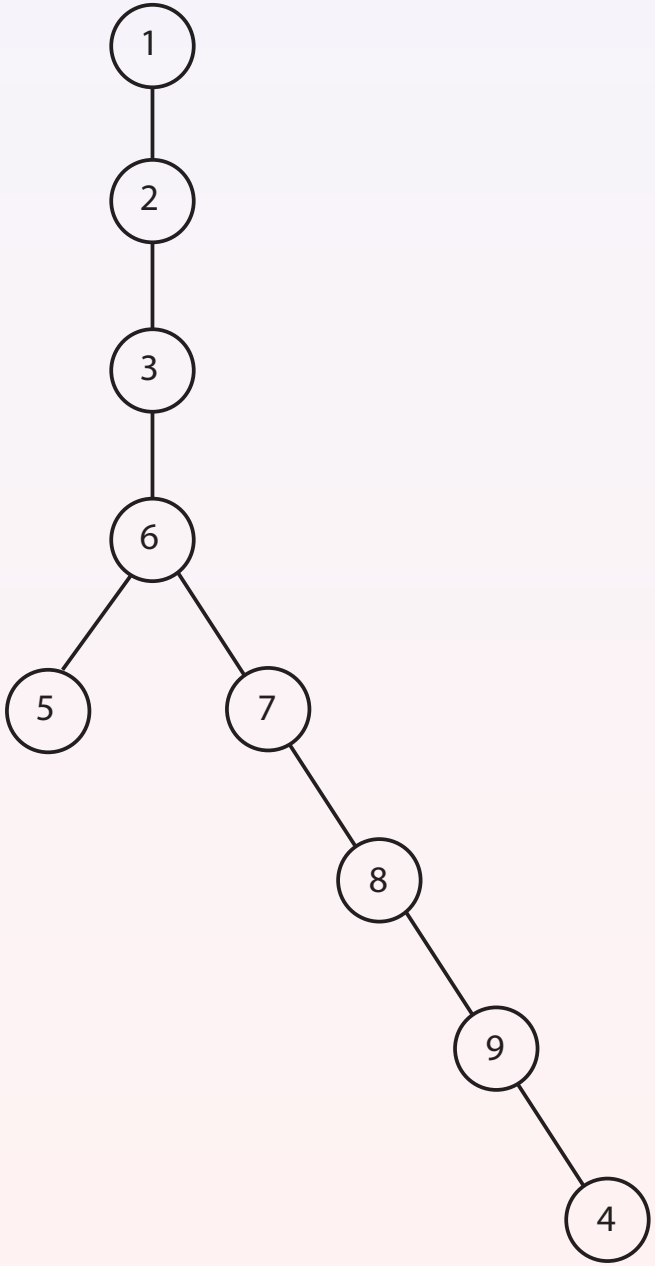
Nesouvislý graf:

```
def DFS(G, u):
    s = ['N'] * (len(G)+1)          #All vertices are new
    p = [None] * (len(G)+1)         #Without predecessors
    for u, value in G.items():      #Process all new nodes
        if s[u] == 'N':
           DFSR(G, s, p, u)         #Run DFS for disconnected graph
```

Rekurzivní procedura:

```
def DFSR(G, s, p, u):
    s[u] = 'O'                      #Set node as open
    for v in G[u]:                  #Browse all edges from u
        if s[v] == 'N':             #For each new node
            p[v] = u                #Set predeccessor
            DFSR (G, s, p, v)       #Browse its neighbors
    s[u] = 'C'                      #Node u is closed
```

27. Výsledný DF strom



28. Použití BFS/DFS

Jedny z nejčastěji používaných algoritmů.

Častá aplikace i pro problémy s grafy zdánlivě nesouvisejícími.

Nejčastější použití:

- *Optimalizační strategie:*
  Heuristika, TSP (přibližné řešení), turnajová schémata, procházení složek/podsložek.
- *Stolní hry:*
  Šachy, dáma, GO, piškvorky + doplněno efektivnějšími strategiemi.
- *Hlavolamy:*
  Magické čtverce, SUDOKU, křížovky, Rubikova kostka, bludiště.
- *Dopravní a logistické problémy*
  Existence cesty, optimální cesty, vzdálenosti centra/pobočky, údržba silnic, elektrifikace.

29. Piškvorky, stavový strom

